

Parallel Change

13 May 2014



Danilo Sato

◇ EVOLUTIONARY DESIGN
◇ API DESIGN
◇ REFACTORING

Making a change to an interface that impacts all its consumers requires two thinking modes: implementing the change itself, and then updating all its usages. This can be hard when you try to do both at the same time, especially if the change is on a PublishedInterface with multiple or external clients.

Parallel change, also known as **expand and contract**, is a pattern to implement backward-incompatible changes to an interface in a safe manner, by breaking the change into three distinct phases: expand, migrate, and contract.

To understand the pattern, let's use an example of a simple Grid class that stores and provides information about its cells using a pair of x and y integer coordinates. Cells are stored internally in a two-dimensional array and clients can use the `addCell()`, `fetchCell()` and `isEmpty()` methods to interact with the grid.

```

class Grid {
    private Cell[][] cells;
    ...

    public void addCell(int x, int y, Cell cell) {
        cells[x][y] = cell;
    }

    public Cell fetchCell(int x, int y) {
        return cells[x][y];
    }

    public boolean isEmpty(int x, int y) {
        return cells[x][y] == null;
    }
}

```

As part of refactoring, we detect that `x` and `y` are a DataClump and decide to introduce a new `Coordinate` class. However, this will be a backwards-incompatible change to clients of the `Grid` class. Instead of changing all the methods and the internal data structure at once, we decide to apply the parallel change pattern.

In the *expand* phase you augment the interface to support both the old and the new versions. In our example, we introduce a new `Map<Coordinate, Cell>` data structure and the new methods that can receive `Coordinate` instances without changing the existing code.

```

class Grid {
    private Cell[][] cells;
    private Map<Coordinate, Cell> newCells;
    ...

    public void addCell(int x, int y, Cell cell) {
        cells[x][y] = cell;
    }

    public void addCell(Coordinate coordinate, Cell cell) {
        newCells.put(coordinate, cell);
    }

    public Cell fetchCell(int x, int y) {
        return cells[x][y];
    }

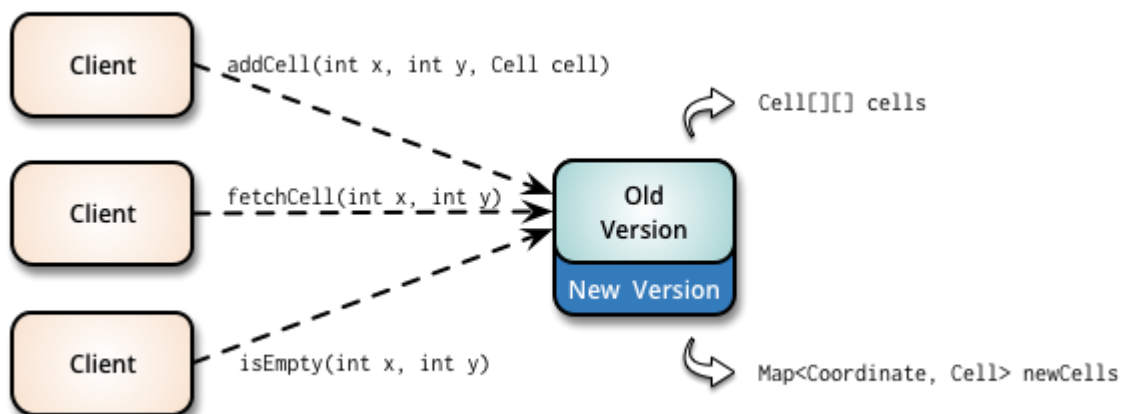
    public Cell fetchCell(Coordinate coordinate) {
        return newCells.get(coordinate);
    }

    public boolean isEmpty(int x, int y) {
        return cells[x][y] == null;
    }

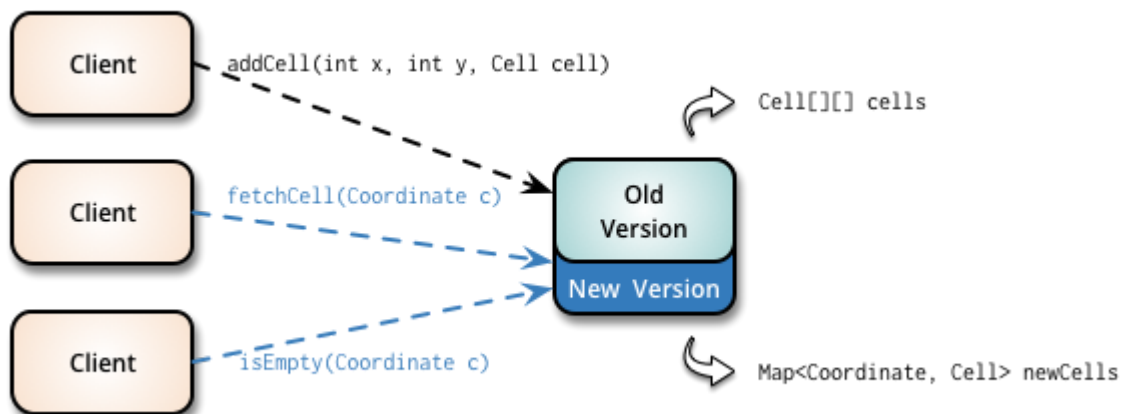
    public boolean isEmpty(Coordinate coordinate) {
        return !newCells.containsKey(coordinate);
    }
}

```

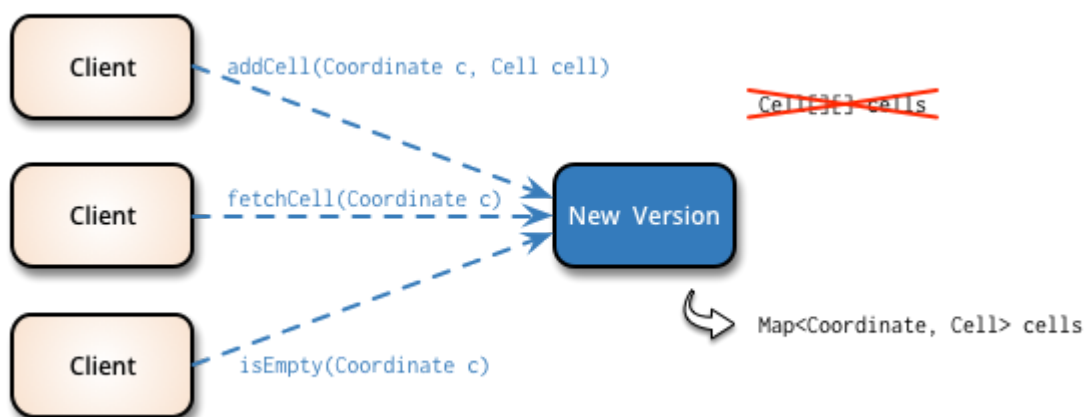
Existing clients will continue to consume the old version, and the new changes can be introduced incrementally without affecting them.



During the *migrate* phase you update all clients using the old version to the new version. This can be done incrementally and, in the case of external clients, this will be the longest phase.



Once all usages have been migrated to the new version, you perform the *contract* phase to remove the old version and change the interface so that it only supports the new version.



In our example, since the internal two-dimensional array is not used anymore after the old methods have been deleted, we can safely remove that data structure and rename `newCells` back to `cells`.

```

class Grid {
    private Map<Coordinate, Cell> cells;
    ...

    public void addCell(Coordinate coordinate, Cell cell) {
        cells.put(coordinate, cell);
    }

    public Cell fetchCell(Coordinate coordinate) {
        return cells.get(coordinate);
    }

    public boolean isEmpty(Coordinate coordinate) {
        return !cells.containsKey(coordinate);
    }
}

```

This pattern is particularly useful when practicing ContinuousDelivery because it allows your code to be released in any of these three phases. It also lowers the risk of change by allowing you to migrate clients and to test the new version incrementally.

Even when you have control over all usages of the interface, following this pattern is still useful because it prevents you from spreading breakage across the entire codebase all at once. The migrate phase can be short, but it is an alternative to leaning on the compiler to find all the usages that need to be fixed.

Some example applications of this pattern are:

- *Refactoring*: when changing a method or function signature, especially when doing a Long Term Refactoring or when changing a PublishedInterface. A variant implementation of this pattern during a refactoring is to implement the old method in terms of the new API and use Inline Method to update all usages at once. Delegating the old method to the new method is also a way to break the migrate phase into smaller and safer steps, allowing you to change the internal implementation first before changing the exposed API to clients. This is useful when the migrate phase is longer so you don't have to maintain two separate implementations.
- *Database refactoring*: this is a key component to evolutionary database design. Most database refactorings follow the parallel change pattern, where the migrate phase is the transition period between the original and the new schema, until all database access code has been updated to work with the new schema.

- *Deployments*: deployment techniques such as canary releases and BlueGreenDeployment are applications of the parallel change pattern where you have both old and new versions of the code deployed side by side, and you incrementally migrate users from one version to another, therefore lowering the risk of change. In a microservices architecture, it can also remove the need for complex deployment orchestration of different services due to version dependencies between them.
- *Remote API evolution*: parallel change can be used to evolve a remote API (e.g. a REST web service) when you can't make the change in a backwards compatible manner. This is an alternative to using an explicit version in the exposed API. You can apply the pattern when making a change to the payload accepted or returned by the API on a given endpoint, or you can introduce a new endpoint to distinguish between the old and new versions. In the case of using parallel change in the same endpoint, following Postel's Law is a good technique to avoid consumers breaking when the payload is expanded.

During the migrate phase, a FeatureFlag can be used to control which version of the interface is used. A feature toggle on the client side allows it to be forward-compatible with the new version of the supplier, which decouples the release of the supplier from the client.

When implementing BranchByAbstraction, parallel change is a good way to introduce the abstraction layer between the clients and the supplier. It is also an alternative way to perform a large-scale change without introducing the abstraction layer as a seam for replacement on the supplier side. However, when you have a large number of clients, using branch by abstraction is a better strategy to narrow the surface of change and reduce confusion during the migrate phase.

The downside of using parallel change is that during the migrate phase the supplier has to support two different versions, and clients could get confused about which version is new versus old. If the contract phase is not executed you might end up in a worse state than you started, therefore you need discipline to finish the transition successfully. Adding deprecation notes, documentation or TODO notes might help inform clients and other developers working on the same codebase about which version is in the process of being replaced.

Further Reading

Industrial Logic's refactoring album documents and demonstrates an example of performing a parallel change.

Acknowledgements

This technique was first documented as a refactoring strategy by Joshua Kerievsky in 2006 and presented in his talk [The Limited Red Society](#) presented at the Lean Software and Systems Conference in 2010.

Thanks to Joshua Kerievsky for giving feedback on the first draft of this post. Also thanks to many Thoughtworks colleagues for their feedback: Greg Dutcher, Badrinath Janakiraman, Praful Todkar, Rick Carragher, Filipe Esperandio, Jason Yip, Tushar Madhukar, Pete Hodgson, and Kief Morris.



© Martin Fowler | Disclosures